EXPERIMENT 17

IMPLEMENTATION OF HASH FUNCTION – MD5

AIM

To implement the MD5 (Message-Digest Algorithm 5) hash function in C and generate the 128-bit message digest for a given input message.

ALGORITHM

1. Accept the input message from the user.
2. Convert the message into a sequence of bytes.
3. Append a single 1 bit to the message, followed by 0 bits until the message length is 448 bits modulo 512.
4. Append the original message length as a 64-bit value.
5. Divide the resulting message into 512-bit blocks.
6. Initialize four 32-bit buffers:
    - A
    - B
    - C
    - D
7. Process each 512-bit block through 64 operations, arranged into four rounds of 16 operations.
8. Use the four nonlinear functions:
    - F
    - G
    - H
    - I
9. Add the results to the initial buffer values.
10. After processing all blocks, concatenate A, B, C, and D.
11. The resulting 128-bit value is the MD5 hash/digest.

PROGRAM:

```
#include <stdio.h>

#include <stdint.h>

#include <string.h>

#include <stdlib.h>


#define LEFTROTATE(x, c) (((x) << (c)) | ((x) >> (32 - (c))))


uint32_t K[64] = {

    0xd76aa478, 0xe8c7b756, 0x242070db, 0xc1bdceee,
```

```
    0xf57c0faf, 0x4787c62a, 0xa8304613, 0xfd469501,
    0x698098d8, 0x8b44f7af, 0xffff5bb1, 0x895cd7be,
    0x6b901122, 0xfd987193, 0xa679438e, 0x49b40821,
    0xf61e2562, 0xc040b340, 0x265e5a51, 0xe9b6c7aa,
    0xd62f105d, 0x02441453, 0xd8a1e681, 0xe7d3fbc8,
    0x21e1cde6, 0xc33707d6, 0xf4d50d87, 0x455a14ed,
    0xa9e3e905, 0xfcefa3f8, 0x676f02d9, 0x8d2a4c8a,
    0xfffa3942, 0x8771f681, 0x6d9d6122, 0xfde5380c,
    0xa4beea44, 0x4bdecfa9, 0xf6bb4b60, 0xbebfbc70,
    0x289b7ec6, 0xeaa127fa, 0xd4ef3085, 0x04881d05,
    0xd9d4d039, 0xe6db99e5, 0x1fa27cf8, 0xc4ac5665,
    0xf4292244, 0x432aff97, 0xab9423a7, 0xfc93a039,
    0x655b59c3, 0x8f0ccc92, 0xffeff47d, 0x85845dd1,
    0x6fa87e4f, 0xfe2ce6e0, 0xa3014314, 0x4e0811a1,
    0xf7537e82, 0xbd3af235, 0x2ad7d2bb, 0xeb86d391
};

uint32_t shift[64] = {
     7, 12, 17, 22,  7, 12, 17, 22,
     7, 12, 17, 22,  7, 12, 17, 22,
     5,  9, 14, 20,  5,  9, 14, 20,
     5,  9, 14, 20,  5,  9, 14, 20,
     4, 11, 16, 23,  4, 11, 16, 23,
     4, 11, 16, 23,  4, 11, 16, 23,
     6, 10, 15, 21,  6, 10, 15, 21,
     6, 10, 15, 21,  6, 10, 15, 21
};

void md5(const unsigned char *message, size_t len)
{
```

```
uint32_t a0 = 0x67452301;
uint32_t b0 = 0xefcdab89;
uint32_t c0 = 0x98badcfe;
uint32_t d0 = 0x10325476;

uint64_t bitLen = (uint64_t)len * 8;

/* Calculate padded length */
size_t newLen = len + 1;

while (newLen % 64 != 56)
    newLen++;

unsigned char *data =
    (unsigned char *)calloc(newLen + 8, 1);

if (data == NULL)
{
    printf("Memory allocation failed.\n");
    return;
}

/* Copy original message */
memcpy(data, message, len);

/* Append one bit followed by zeros */
data[len] = 0x80;

/* Append original length in bits */
memcpy(data + newLen, &bitLen, 8);
```

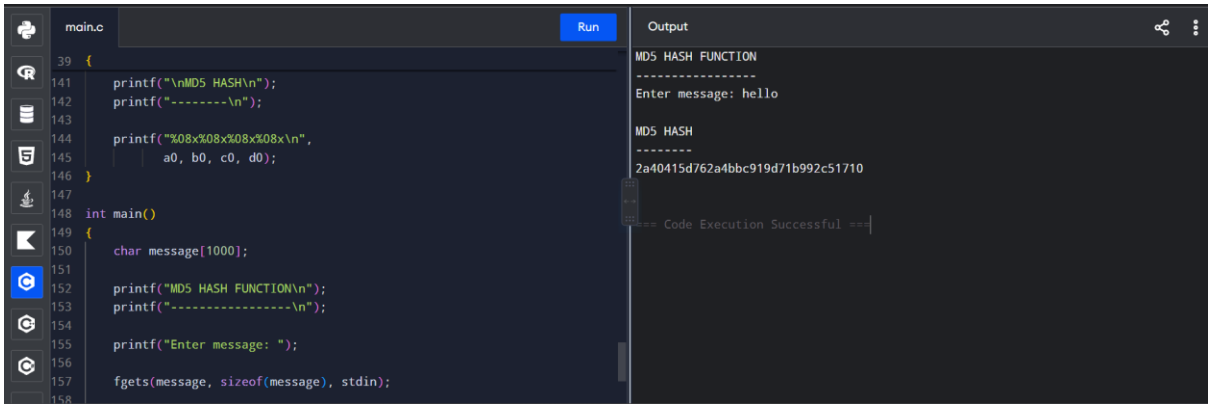
```
/* Process 512-bit blocks */
for (size_t offset = 0;
     offset < newLen;
     offset += 64)
{
    uint32_t M[16];

    /* Convert block to 32-bit words */
    for (int i = 0; i < 16; i++)
    {
        M[i] =
            ((uint32_t)data[offset + i * 4]) |
            ((uint32_t)data[offset + i * 4 + 1] << 8) |
            ((uint32_t)data[offset + i * 4 + 2] << 16) |
            ((uint32_t)data[offset + i * 4 + 3] << 24);
    }

    uint32_t A = a0;
    uint32_t B = b0;
    uint32_t C = c0;
    uint32_t D = d0;

    /* 64 MD5 operations */
    for (int i = 0; i < 64; i++)
    {
        uint32_t F;
        int g;

        if (i < 16)
```

```
{
    F = (B & C) | ((~B) & D);
    g = i;
}
else if (i < 32)
{
    F = (D & B) | ((~D) & C);
    g = (5 * i + 1) % 16;
}
else if (i < 48)
{
    F = B ^ C ^ D;
    g = (3 * i + 5) % 16;
}
else
{
    F = C ^ (B | (~D));
    g = (7 * i) % 16;
}

uint32_t temp = D;

D = C;
C = B;

B = B + LEFTROTATE(
    A + F + K[i] + M[g],
    shift[i]
);
```

```
            A = temp;
        }

        a0 += A;
        b0 += B;
        c0 += C;
        d0 += D;
    }

    free(data);

    printf("\nMD5 HASH\n");
    printf("--------\n");

    printf("%08x%08x%08x%08x\n",
           a0, b0, c0, d0);
}

int main()
{
    char message[1000];

    printf("MD5 HASH FUNCTION\n");
    printf("-----------------\n");

    printf("Enter message: ");

    fgets(message, sizeof(message), stdin);

    /* Remove newline */
```

```
    message[strcspn(message, "\n")] = '\0';

    md5(
        (unsigned char *)message,
        strlen(message)
    );

    return 0;
}
```

OUTPUT:



RESULT:

Thus, the MD5 hash function was successfully implemented in C, and the 128-bit message digest was generated for the given input message.